StaffSync

System Architecture & Entity–Relationship Diagram

Project overview

StaffSync is a web-based HR management application with a React/Vite frontend and a Node.js/Express API backed by MongoDB through Mongoose. The documented source includes authentication and role middleware, employee management, workforce features, communications, notifications, dashboard endpoints, and system routes.

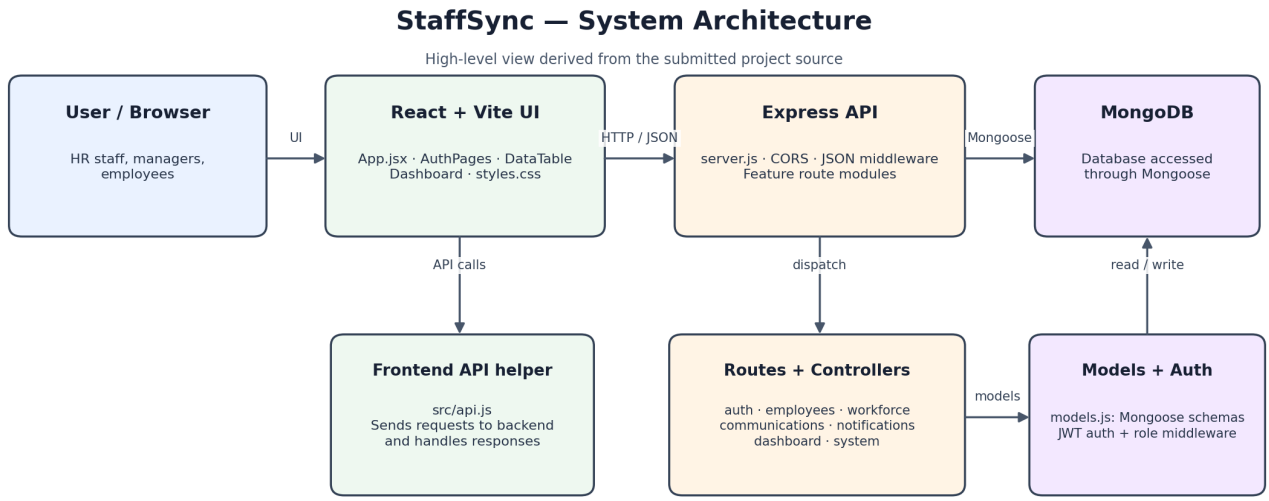
Technology stack

Frontend: React, Vite, JavaScript, CSS
Backend: Node.js, Express.js, REST-style HTTP endpoints
Database: MongoDB with Mongoose schemas
Authentication: JSON Web Tokens (JWT); password hashing with bcryptjs in the seed/auth workflow
Version control: Git and GitHub

Prepared from the submitted StaffSync_Project.zip source. Diagrams reflect the implementation visible in backend/server.js and backend/models.js.

1. System Architecture

The application is organized as a browser-based client and an Express API. The frontend uses a shared API helper to communicate with backend routes. Feature routes dispatch to controllers; controllers use Mongoose models to read and write MongoDB documents. Authentication middleware validates bearer JWTs, while role middleware restricts selected actions.



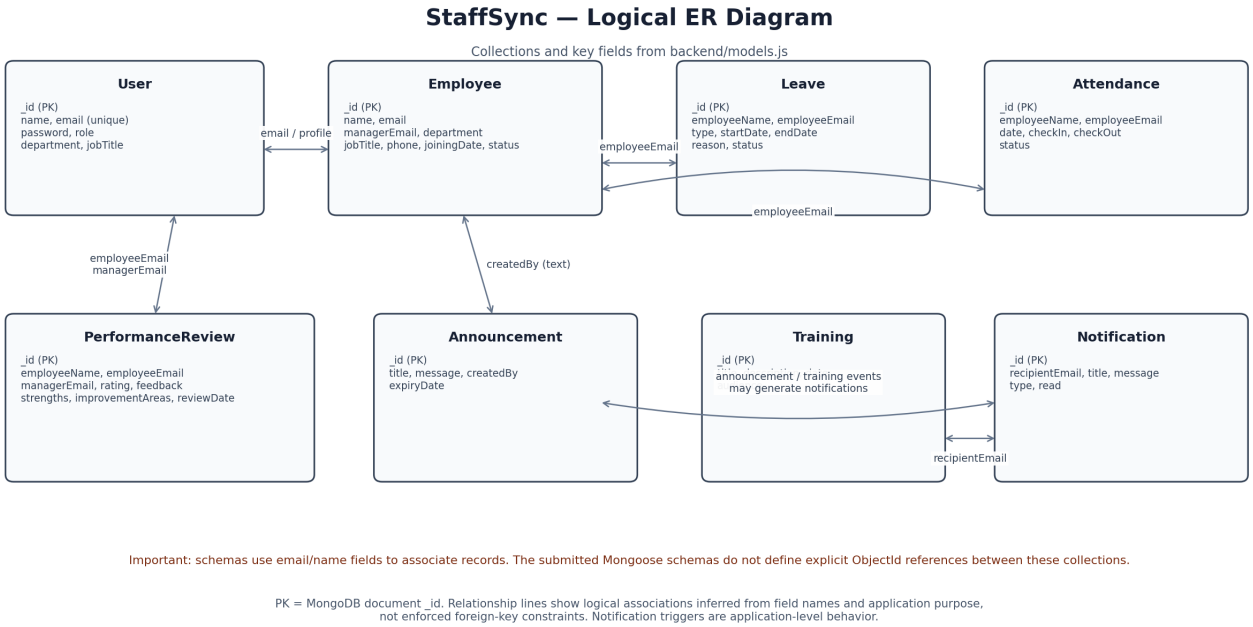
Request flow

1. A user interacts with the React interface in the browser.
2. Frontend code calls the API helper in `frontend/src/api.js`, which sends HTTP requests and receives JSON responses.
3. Express middleware parses JSON and enables CORS; route modules select the relevant endpoint/controller.
4. Protected routes use JWT authentication and, where configured, role checks.
5. Controllers work with the Mongoose models; MongoDB stores application documents.
6. The API returns a response that the frontend uses to update the view.

Backend route groups visible in the project: authentication, employees, workforce, communications, notifications, dashboard, and system. Application startup and database connection are handled by backend/server.js.

2. Entity–Relationship Diagram

The diagram below lists the eight Mongoose models defined in backend/models.js. It is a logical ER view: the fields suggest associations between users/employees, leave and attendance records, reviews, announcements/training, and notifications.



Interpretation note

Each collection has MongoDB's default `_id` identifier. The submitted schemas store several associations as email/name strings (for example, employeeEmail and managerEmail) rather than explicit Mongoose ObjectId references. Therefore, the lines in this diagram indicate logical associations inferred from field names and application purpose; they are not database-enforced foreign keys. Verify application logic before treating any association as a guaranteed one-to-many constraint.

3. Data Model Summary

The following summary is based on the fields declared in backend/models.js. MongoDB/Mongoose also adds createdAt and updatedAt timestamps to these schemas where timestamps are enabled.

Collection	Purpose	Important fields
User	Login identity and role/profile information.	name, email (unique), password, role, department, jobTitle
Employee	Employee directory and employment details.	name, email, managerEmail, department, jobTitle, phone, joiningDate, status
Leave	Leave requests and approval state.	employeeName, employeeEmail, type, startDate, endDate, reason, status
Attendance	Daily attendance and time tracking.	employeeName, employeeEmail, date, checkIn, checkOut, status
PerformanceReview	Manager feedback and employee rating.	employeeName, employeeEmail, managerEmail, rating, feedback, strengths, improvementAreas, reviewDate
Announcement	Company messages and expiry.	title, message, createdBy, expiryDate
Training	Training session metadata and intended audience.	title, description, date, audience
Notification	Per-recipient alerts and read state.	recipientEmail, title, message, type, read

4. Main Logical Associations

• Employee records are associated with leave and attendance records through employeeEmail and employeeName fields.
• PerformanceReview records identify the reviewed employee and manager through employeeEmail and managerEmail.
• User accounts and employee directory records both store email/profile fields; the schema does not define an explicit foreign-key link between them.
• Notification records target a recipient through recipientEmail and categorize messages as leave, announcement, training, or employee.
• Announcement.createdBy is a text field, and Training.audience is a text description; neither is modeled as a formal reference to another collection.

5. Security and Implementation Notes

• JWT authentication middleware reads a bearer token from the Authorization header and verifies it using JWT_SECRET.
• Role middleware can reject requests when the authenticated user's role is not in the allowed role list; each route's actual middleware configuration determines which operations are protected.
• The backend requires MONGO_URI and JWT_SECRET at startup, configured through backend/.env locally. The committed backend/.env.example should contain placeholders only.
• Passwords are hashed with bcryptjs in the seed script before demo users are inserted. Review the auth controller to verify the full registration/login behavior and password handling before production use.
• The diagram is architectural documentation, not a security audit. Production readiness also depends on validation, authorization checks, secret management, error handling, backups, and automated tests.

Source files reviewed

backend/server.js · backend/models.js · backend/auth.js · backend/routes/*.js · backend/controllers/*.js · frontend/src/App.jsx · frontend/src/api.js · README.md

End of document